Exam: Design Patterns, Polymorphism & Interfaces

Name: _____
Date: Juni 2026
Time: 60 minutes
Points: 100

Question 1: Observer Pattern (30 Points)

Task Description

You are developing a game with a notification system. When important events happen in the game (player levels up, boss spawns, treasure found), multiple game components need to be notified: - **HUD Display**: Shows notifications on screen - **Sound Manager**: Plays appropriate sound effects - **Achievement System**: Checks if any achievements are unlocked - **Logger**: Records events to a log file

Requirements: - New events can occur at any time during gameplay - Multiple components should be notified automatically when an event occurs - It should be easy to add new observers (e.g., a Statistics Tracker) without modifying existing code - Observers should be able to subscribe and unsubscribe from notifications dynamically

Your Task (30 Points)

a) **(20 Points)** Draw a complete UML class diagram using the **Observer Pattern** to solve this problem. Your diagram must include:
   - All necessary classes and interfaces
   - Relationships between classes (use appropriate UML notation)
   - Important methods and attributes (e.g., attach(), detach(), notify())
   - Proper naming conventions

b) **(10 Points)** Explain in 2-3 sentences why the Observer Pattern is the appropriate choice for this scenario and what problem it solves.

Question 2: Polymorphism with Interfaces (40 Points)

Given Class Hierarchy

```
interface Driveable {
    void accelerate();
    void brake();
    int getCurrentSpeed();
}

class Vehicle {
    protected String brand;
    protected int speed;

    public Vehicle(String brand) {
        this.brand = brand;
        this.speed = 0;
    }

    public void start() {
        System.out.println("Vehicle starting");
    }
```

```
    public void stop() {
        System.out.println("Vehicle stopping");
        speed = 0;
    }

    public String getBrand() {
        return brand;
    }
}

class Car extends Vehicle implements Driveable {
    private int numberOfDoors;

    public Car(String brand, int doors) {
        super(brand);
        this.numberOfDoors = doors;
    }

    @Override
    public void start() {
        System.out.println("Car engine starting");
    }

    @Override
    public void accelerate() {
        speed += 10;
        System.out.println("Car accelerating to " + speed + " km/h");
    }

    @Override
    public void brake() {
        speed -= 10;
        if (speed < 0) speed = 0;
        System.out.println("Car braking to " + speed + " km/h");
    }

    @Override
    public int getCurrentSpeed() {
        return speed;
    }

    public void openTrunk() {
        System.out.println("Trunk opening");
    }

    public int getNumberOfDoors() {
        return numberOfDoors;
    }
}

class Bicycle extends Vehicle {
    private int numberOfGears;
```

```
    public Bicycle(String brand, int gears) {
        super(brand);
        this.numberOfGears = gears;
    }

    @Override
    public void start() {
        System.out.println("Start pedaling");
    }

    public void ringBell() {
        System.out.println("Ring ring!");
    }

    public int getNumberOfGears() {
        return numberOfGears;
    }
}
```

Questions

a) **(5 Points)** Draw a simple UML class diagram showing the hierarchy of the Driveable interface, Vehicle class, Car, and Bicycle. Include the implements and extends relationships.

b) **(15 Points)** Consider the following code:

```
Vehicle v1 = new Car("BMW", 4);
Driveable d1 = new Car("Audi", 2);
Car c1 = new Car("Tesla", 4);
Vehicle v2 = new Bicycle("Trek", 21);
```

Fill in the table:

Code Statement	Compiles? (Yes/No)	Executes? (Yes/No)	Which object's method is called?	Output/Explanation
`v1.start();`				
`v1.accelerate();`				
`d1.accelerate();`				
`d1.getBrand();`				
`c1.openTrunk();`				
`v2.ringBell();`				
`((Car)v1).openTrunk();`				
`((Driveable)v2).brake();`				

c) **(10 Points)** Explain the following concepts with examples from the code above:
- **Method Overriding** (5 Points)
- **Interface Implementation** (5 Points)

d) **(10 Points)** What is the difference between:

```
Vehicle v = new Car("Tesla", 4);
Driveable d = new Car("Tesla", 4);
Car c = new Car("Tesla", 4);
```

Which methods can be called on `v`, `d`, and `c`? Explain the concept of compile-time vs. runtime types.

Question 3: Interfaces (30 Points)

a) **(15 Points)** What is an interface in object-oriented programming? Your answer should include:
- A definition (3 points)
- Two advantages of using interfaces (6 points)
- One example of when you would use an interface (6 points)

b) **(15 Points)** Given the following interface:

```
interface Rechargeable {
    void recharge();
    int getBatteryPercentage();
    void setBatteryPercentage(int percentage);
    boolean isFullyCharged();
}
```

Write a class `ElectricBike` that: - Extends the `Vehicle` class from Question 2 - Implements the `Rechargeable` interface - Include all necessary constructors and method implementations

---

Good Luck!
